Referencia de la API REST v3

Nimbus Cloud - Gestión de dispositivos, telemetría y firmware

Código: NE-210 **Versión:** 3.4 **Fecha:** 2026-10-05

Público objetivo: Desarrolladores e integradores

1. Introducción

La API REST de Nimbus Cloud permite administrar dispositivos Nimbus Edge, consultar telemetría histórica, enviar comandos remotos, gestionar campañas de firmware y suscribirse a eventos mediante webhooks.

La URL base de la versión actual es `https://api.nimbus-edge.example.com/v3`. Todas las solicitudes y respuestas usan JSON codificado en UTF-8 y todas las fechas se expresan en formato ISO 8601 en UTC, por ejemplo `2026-10-05T14:30:00Z`.

> **Advertencia:** La API v2 está obsoleta y dejará de funcionar el **31 de diciembre de 2026**. Los clientes que aún la utilicen deben migrar a v3 antes de esa fecha. Consulte la sección 9.

2. Autenticación

La API v3 usa **OAuth 2.0 con el flujo client credentials**. El cliente obtiene un token de acceso en `POST /v3/auth/token` y lo envía en la cabecera `Authorization: Bearer <token>` en cada solicitud. Los tokens tienen una vigencia de **3600 segundos** (1 hora) y no se renuevan automáticamente: al expirar debe solicitarse uno nuevo.

```
curl -X POST https://api.nimbus-edge.example.com/v3/auth/token \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "client_id=$NIMBUS_CLIENT_ID" \
  -d "client_secret=$NIMBUS_CLIENT_SECRET" \
  -d "scope=devices:read telemetry:read"

{
  "access_token": "eyJhbGciOiJFUzI1NiJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "devices:read telemetry:read"
}
```

2.1 Scopes

Scope	Permite
`devices:read`	Listar dispositivos y consultar su detalle
`devices:write`	Modificar la configuración remota y enviar comandos
`telemetry:read`	Consultar telemetría histórica
`alarms:read`	Listar alarmas activas e históricas
`firmware:manage`	Crear y consultar campañas de actualización de firmware
`webhooks:manage`	Crear, listar y eliminar suscripciones de webhooks

Si el token no incluye el scope requerido por el endpoint, la API responde `403` con el código `insufficient_scope`.

3. Convenciones generales

- **Paginación:** los listados usan cursor opaco. Parámetros `page_size` (por defecto 50, máximo **200**) y `cursor`. La respuesta incluye `next_cursor`, que es nulo en la última página.

- **Idempotencia:** las solicitudes `POST` aceptan la cabecera `Idempotency-Key`. Repetir una solicitud con la misma clave durante 24 horas devuelve la respuesta original sin ejecutar la operación de nuevo.
- **Identificadores:** `device_id` tiene el formato `dev_` seguido de 12 caracteres alfanuméricos, por ejemplo `dev_8f3k2m9x1q4z`.
- **Compresión:** la API acepta `Accept-Encoding: gzip`.

4. Límites de uso

Cada API key tiene un límite de **600 solicitudes por minuto**, con ráfagas de hasta 20 solicitudes por segundo. Cuando se supera el límite, la API responde `429` con el código `rate_limited` e incluye la cabecera `Retry-After` con los segundos que se deben esperar.

Todas las respuestas incluyen las cabeceras `X-RateLimit-Limit` y `X-RateLimit-Remaining`. Se recomienda aplicar reintentos con espera exponencial y no reintentar errores `4xx` distintos de `429`.

5. Endpoints

Método	Ruta	Scope	Descripción
GET	`/v3/devices`	devices:read	Lista los dispositivos del tenant
GET	`/v3/devices/{device_id}`	devices:read	Detalle y estado de un dispositivo
PUT	`/v3/devices/{device_id}/config`	devices:write	Reemplaza la configuración remota
POST	`/v3/devices/{device_id}/commands`	devices:write	Envía un comando al dispositivo
GET	`/v3/devices/{device_id}/telemetry`	telemetry:read	Consulta telemetría histórica
GET	`/v3/alarms`	alarms:read	Lista alarmas por severidad y estado
POST	`/v3/firmware/campaigns`	firmware:manage	Crea una campaña de actualización
GET	`/v3/firmware/campaigns/{campaign_id}`	firmware:manage	Estado de una campaña
POST	`/v3/webhooks`	webhooks:manage	Crea una suscripción de webhook

5.1 Listar dispositivos

`GET /v3/devices` admite los filtros `status` (`online`, `offline`, `unprovisioned`), `model` (`GX-200`, `GX-200L`) y `firmware_version`. Ejemplo: `/v3/devices?status=offline&page_size=100`.

5.2 Enviar comandos

`POST /v3/devices/{device_id}/commands` encola un comando para el dispositivo y responde `202 Accepted` con un `command_id`. Los comandos disponibles son `reboot`, `sync_config`, `collect_diagnostics` y `set_relay`. El dispositivo debe estar en línea; si no confirma la ejecución dentro de `timeout_s` (por defecto **120 segundos**), el comando pasa al estado `expired`.

```
POST /v3/devices/dev_8f3k2m9x1q4z/commands
{
  "command": "set_relay",
  "params": { "relay": 1, "state": "on" },
  "timeout_s": 120
}
```

5.3 Consultar telemetría

`GET /v3/devices/{device_id}/telemetry` recibe los parámetros `from` y `to` (obligatorios), `metric`, `aggregation` (`avg`, `min`, `max`, `sum`) e `interval` (`1m`, `5m`, `1h`). Cada solicitud admite un rango máximo de **31 días** y devuelve hasta **10 000 puntos**; si el resultado es mayor, la respuesta incluye `next_cursor`.

Métricas de sistema disponibles: `cpu_temp_c`, `cpu_load_pct`, `mem_used_pct`, `disk_used_pct` y `wan_rssi_dbm` (solo GX-200L). Las métricas definidas por el usuario a partir de registros Modbus se consultan por el nombre configurado en el mapa de registros.

5.4 Campañas de firmware

`POST /v3/firmware/campaigns` crea una campaña con los campos `target_version`, `device_filter` y `rollout`. Dentro de `rollout` se definen `batch_percent` (por defecto 10), `pause_minutes` (por defecto 30) y `abort_failure_rate` (por defecto 5). Si ya existe una campaña activa sobre los mismos dispositivos, la API responde `409` con el código `campaign_conflict`. El procedimiento completo se describe en el documento NE-410.

6. Webhooks

Los webhooks notifican eventos a una URL HTTPS del cliente. Los eventos disponibles son `device.online`, `device.offline`, `alarm.raised`, `alarm.cleared` y `firmware.completed`.

- Cada entrega incluye la cabecera `X-Nimbus-Signature` con una firma **HMAC-SHA256** del cuerpo, calculada con el secreto de la suscripción.
- El receptor debe responder con un código `2xx` en menos de **10 segundos**.
- Si la entrega falla, Nimbus Cloud reintenta hasta **5 veces** con espera exponencial de 1, 2, 4, 8 y 16 minutos. Agotados los reintentos, la entrega se marca como fallida y se genera una alarma de integración.

> **Nota:** Valide siempre la firma antes de procesar el evento y compare con una función de tiempo constante para evitar ataques de temporización.

7. Errores HTTP

HTTP	Código	Descripción	Acción recomendada
400	`invalid_request`	Solicitud mal formada o parámetro faltante	Revisar el cuerpo y los parámetros
401	`invalid_token`	Token ausente, expirado o inválido	Solicitar un token nuevo
403	`insufficient_scope`	El token no tiene el scope necesario	Solicitar el scope requerido
404	`device_not_found`	El dispositivo no existe en el tenant	Verificar el `device_id`
409	`campaign_conflict`	Ya existe una campaña activa para esos equipos	Esperar o cancelar la campaña vigente
422	`validation_failed`	Valores fuera de rango o inválidos	Corregir según el detalle de `errors`
429	`rate_limited`	Se superó el límite de 600 solicitudes por minuto	Esperar `Retry-After` segundos
503	`maintenance`	Plataforma en mantenimiento programado	Reintentar más tarde

8. Ejemplo completo

El siguiente flujo obtiene los dispositivos desconectados y envía un reinicio al primero de ellos.

```
# 1) Listar dispositivos fuera de línea
curl -H "Authorization: Bearer $TOKEN" \
  "https://api.nimbus-edge.example.com/v3/devices?status=offline&page_size=10"

# 2) Enviar reinicio
curl -X POST -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: 6b1f0c52-reboot-001" \
  -H "Content-Type: application/json" \
  -d '{"command":"reboot","timeout_s":120}' \
  "https://api.nimbus-edge.example.com/v3/devices/dev_8f3k2m9x1q4z/commands"
```

9. Migración de v2 a v3

Aspecto	API v2	API v3
Autenticación	API key estática en `X-Api-Key`	OAuth 2.0 client credentials, tokens de 3600 s
Paginación	Parámetros `page` y `per_page`	Cursor opaco con `page_size` (máx. 200)
Límite de uso	300 solicitudes por minuto	600 solicitudes por minuto
Webhooks	No disponibles	Disponibles con firma HMAC-SHA256
Formato de fechas	Epoch en segundos	ISO 8601 en UTC
Fin de soporte	31 de diciembre de 2026	Vigente